
EXPERIMENTAL STUDY FOR SMISHING DETECTION IN SMS MESSAGES

Dr. Subhas Barman

Head of Dept., Research Mentor
Department of Computer Science and Engineering
Jalpaiguri Government Engineering College, Jalpaiguri
West Bengal
subhas.barman@cse.jgec.ac.in

Anurag Mukherjee

Department of Computer Science and Engineering
Jalpaiguri Government Engineering College, Jalpaiguri
West Bengal
amdgp16@gmail.com

Rebanta Sarkar

Department of Computer Science and Engineering
Jalpaiguri Government Engineering College, Jalpaiguri
West Bengal
sarkarrebanta.2004@gmail.com

Arijit Roy

Department of Computer Science and Engineering
Jalpaiguri Government Engineering College, Jalpaiguri
West Bengal
aroy6934@gmail.com

February 12, 2026

ABSTRACT

SMS spam and smishing detection remains a critical challenge in automated content filtering systems, often addressed using black-box deep learning models. This work presents a unified comparison of classical machine learning, ensemble, and deep learning approaches for three-class SMS classification into ham, spam, and smishing. Classical and ensemble models use TF-IDF features, while deep models include a BiLSTM and a transformer-based BERT for semantic representation learning. Experiments are conducted on a balanced public dataset [1] under a consistent preprocessing and evaluation pipeline, comparing Logistic Regression, Multinomial Naïve Bayes, Linear SVM, Random Forest, XGBoost, BiLSTM, and BERT. Results show that Linear SVM, Random Forest, and BiLSTM achieve the best overall performance, while simpler models remain competitive, highlighting trade-offs between accuracy, interpretability, and computational cost. This study offers practical guidance for deploying effective SMS spam and smishing detection systems.

1 Introduction

The widespread adoption of digital communication platforms such as email, messaging applications, and social media has led to a substantial increase in malicious messages, commonly referred to as spam and smishing. While spam primarily consists of unsolicited or promotional content, smishing poses a more serious threat by attempting to deceive users into disclosing sensitive information or performing harmful actions. The growing volume and increasing sophistication of such messages make manual moderation impractical, thereby necessitating accurate and automated detection systems capable of operating at scale [2, 3].

Machine learning and deep learning techniques have become the dominant approaches for text-based spam detection due to their ability to learn discriminative patterns from large datasets [4, 5]. However, many existing approaches rely on complex models that function as black boxes, offering limited interpretability and increased computational cost. Recent work in related domains such as cyberbullying detection has highlighted the importance of interpretability and transparency, for example through the use of Kolmogorov–Arnold Networks (KANs) and context-aware modeling with metadata [6, 7]. These studies demonstrate that explainable and structured models can provide valuable insights beyond raw predictive performance.

Moreover, model selection often involves trade-offs among classification performance, efficiency, and transparency, which are critical factors in real-world deployment scenarios [8, 9]. In addition, a large portion of prior work focuses on binary classification, which is insufficient for practical settings where multiple categories of malicious content, such as spam and smishing, coexist [10].

To address these limitations, this work presents a systematic comparative study of classical machine learning models, ensemble methods, and deep learning architectures for three-class classification of messages into ham, spam, and smishing. We conduct experiments on a balanced public dataset [1] under a unified preprocessing and evaluation framework, enabling a fair and reproducible comparison across different model families. By analyzing performance across multiple metrics, this study aims to highlight the trade-offs between accuracy, computational complexity, and practical deployability, and to provide useful guidance for building effective SMS spam and smishing detection systems, while being informed by recent advances in interpretable and context-aware text classification [6].

2 Related Works

Spam, smishing, and malicious content detection in digital communication platforms and social media has been extensively studied over the past decade. Early approaches primarily relied on classical machine learning techniques such as Support Vector Machines, Naïve Bayes, and Decision Trees, which provided effective and computationally efficient baselines for text filtering and spam detection tasks [10, 3, 11, 12]. While these methods demonstrated reasonable performance, they were limited by their dependence on hand-crafted features and their reduced scalability and robustness when applied to large, diverse, and evolving datasets.

To address these limitations, more recent research has shifted toward deep learning-based methods. Architectures such as Convolutional Neural Networks (CNNs), Long Short-Term Memory (LSTM) networks, and other recurrent models have shown significant improvements by learning discriminative and sequential representations directly from text [4, 5, 13, 14]. These models are particularly effective at capturing contextual and temporal dependencies in messages, which is crucial for distinguishing benign content from deceptive or malicious messages such as smishing. However, despite their strong performance, deep models often introduce higher computational costs and reduced interpretability, which can limit their applicability in real-world, large-scale filtering systems.

Parallel to these developments, ensemble learning techniques have been widely adopted to improve robustness and generalization. Methods such as Random Forests [15] and gradient boosting frameworks like XGBoost [16] have demonstrated strong and stable performance across a variety of text classification and spam detection tasks. Several surveys and critical evaluations have highlighted the effectiveness of these approaches, while also pointing out persistent challenges such as data imbalance, evolving attack strategies, and model generalization across domains [17, 18, 2].

Another important research direction concerns transparency and explainability in text classification systems. The growing interest in Explainable Artificial Intelligence (XAI) has led to methods that aim to provide insights into model predictions and decision-making processes [8, 9]. Although such techniques improve the interpretability of complex models, most practical spam and smishing detection systems still rely on black-box architectures, making it difficult to balance predictive performance with transparency and trustworthiness in deployment settings.

Finally, context-aware modeling and the use of enriched datasets have been shown to further enhance detection performance. Datasets augmented with additional annotations and metadata enable more fine-grained analysis of harmful content and help models better capture semantic and contextual cues [10]. Building on these ideas, recent works have also demonstrated the effectiveness of transformer-based models such as BERT, which leverage large-scale pretraining to learn rich contextual representations [19]. In this work, we build upon these developments by conducting a systematic comparison of classical machine learning models, ensemble methods, and deep learning architectures, including BiLSTM and BERT, under a unified preprocessing and evaluation framework for three-class SMS classification into ham, spam, and smishing.

3 DATA AND ITS PREPROCESSING

The dataset contains 10,191 SMS messages labeled as ham or smishing, along with three auxiliary features indicating the presence of URLs, email addresses, and phone numbers. Figure 1 shows the first ten rows of the dataframe which we are going to work with.

	LABEL	TEXT	URL	EMAIL	PHONE
0	ham	Hi! You just spoke to DEEPAK. We'd like to kno...	No	No	No
1	ham	Yay can't wait to party together!	No	No	No
2	ham	At what time are you coming.	No	No	No
3	smishing	Dear customer your PAY2TMKYC has been expired,...	No	No	Yes
4	ham	Yo you around? A friend of mine's lookin to pi...	No	No	No
5	smishing	Not heard from U4 a while. Call 4 rude chat pr...	No	No	Yes
6	ham	Where are you call me.	No	No	No
7	ham	Anytime...	No	No	No
8	ham	Just wait till end of march when el nino gets ...	No	No	No
9	spam	Get your garden ready for summer with a FREE s...	No	No	No

Figure 1: The first ten rows of dataframe.

No missing values were found, but 2,169 duplicate entries were removed, resulting in 8,022 unique samples. The categorical features were encoded into binary values, and the class labels were mapped to numeric form. Additionally, two statistical features, character length and word count, were extracted from each message.

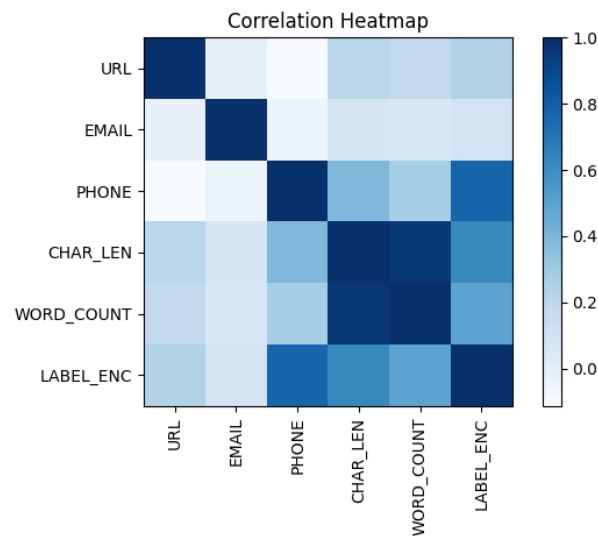


Figure 2: Correlation heatmap of the dataset

4 VISUALIZING THE DATA

Let us check some stats of the data by its visualization. There are a number of bar plots given below.

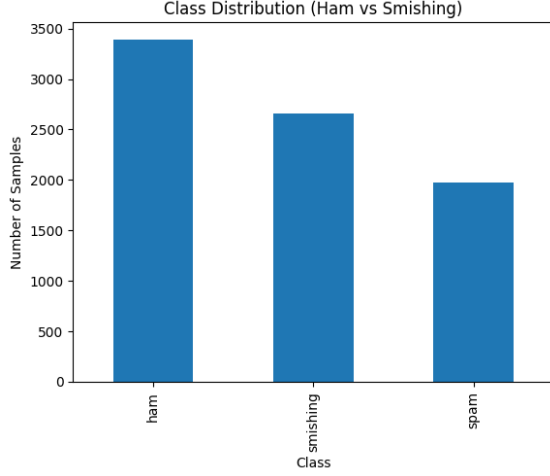


Figure 3: No. of messages from each class

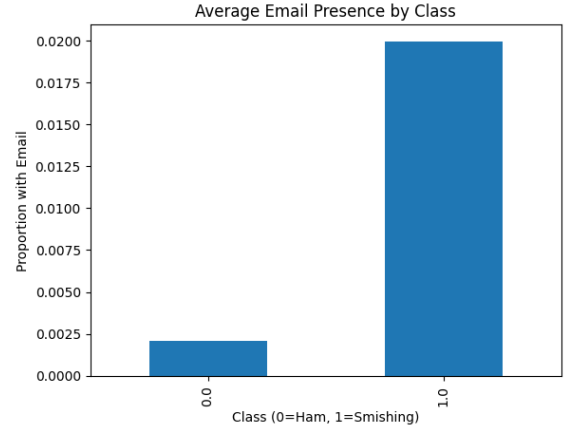


Figure 4: No. of email across message classes

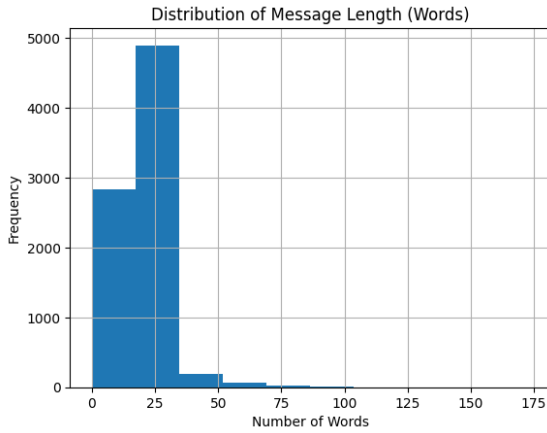


Figure 5: Word-count distribution

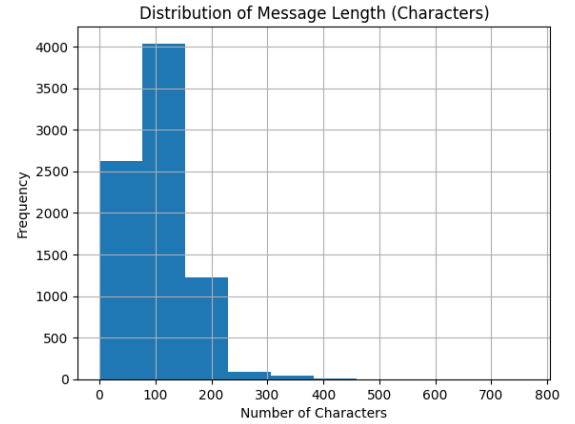


Figure 6: Character length distribution

From Figure 3 we can infer that maximum number of messages are of ham category. Figure 4 shows us that maximum smishing messages are through emails. Figure 5 indicates that most messages contain few words and are generally concise, which is expected for text messages. In Figure 6 we can observe that most SMS messages are short, concentrated in the lower character range. Figure 7 shows us that Turista trains are the maximum in number while Figure 8 shows that promo type tickets have been sold the maximum number of times.

5 Proposed Methodology

This section presents the complete methodology adopted in this work, including the dataset formulation, preprocessing pipeline, feature representations, learning models, optimization procedures, and evaluation metrics. Mathematical formulations are provided to precisely describe each component of the proposed framework.

5.1 Dataset Description

Let the dataset be defined as

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N,$$

where x_i denotes the i -th SMS message and $y_i \in \{1, 2, 3\}$ represents the corresponding class label for *ham*, *spam*, and *smishing*, respectively. Here, N denotes the total number of samples in the dataset. Each input x_i is a short text sequence, while y_i is a categorical variable indicating the semantic intent of the message.

The dataset is approximately balanced across the three classes in order to reduce learning bias toward any dominant category and to ensure stable optimization of multi-class classifiers. To evaluate generalization performance, the dataset is split into disjoint training and test subsets:

$$\mathcal{D}_{train} \cup \mathcal{D}_{test} = \mathcal{D}, \quad \mathcal{D}_{train} \cap \mathcal{D}_{test} = \emptyset,$$

using stratified sampling such that the empirical class prior distribution $P(y)$ is preserved in both subsets. This strategy ensures that performance differences among models are not caused by distributional shifts between training and test data.

5.2 Text Preprocessing and Representation

Each raw message x_i is first normalized by converting all characters to lowercase and removing punctuation, special symbols, and redundant whitespace. This normalization reduces vocabulary size and eliminates superficial variations that do not contribute to semantic meaning. The cleaned message is then tokenized into a sequence of tokens:

$$x_i \rightarrow (w_1, w_2, \dots, w_{T_i}),$$

where w_t denotes the t -th token and T_i is the length of the message in tokens. This representation forms the basis for both sparse and dense feature encodings.

Two types of representations are employed: TF-IDF vectors for classical machine learning models and dense word embeddings for neural models. This dual representation allows a fair comparison between traditional feature-engineering-based methods and representation-learning-based deep models.

5.2.1 TF-IDF Representation

For classical models, each message is represented using the Term Frequency–Inverse Document Frequency (TF-IDF) scheme, which quantifies the importance of a term within a document relative to the entire corpus. For a term t in document d , the TF-IDF weight is defined as:

$$\text{tfidf}(t, d) = \text{tf}(t, d) \cdot \log \left(\frac{N}{\text{df}(t)} \right),$$

where $\text{tf}(t, d)$ denotes the frequency of term t in document d , $\text{df}(t)$ denotes the number of documents in which t appears, and N is the total number of documents. The logarithmic inverse document frequency term down-weights common words and emphasizes rare but potentially discriminative terms.

Each message is thus mapped to a high-dimensional sparse vector:

$$x_i \rightarrow \mathbf{v}_i \in \mathbb{R}^d,$$

where d is the vocabulary size. This vector space model allows linear and tree-based classifiers to operate efficiently in a high-dimensional feature space while preserving important lexical cues.

5.2.2 Neural Input Representation

For deep learning models, each token w_t is mapped to an integer index:

$$x_i \rightarrow (k_1, k_2, \dots, k_{T_i}),$$

where $k_t \in \{1, \dots, V\}$ and V is the vocabulary size. Since neural networks require fixed-length inputs, the sequence is padded or truncated to a predefined length L :

$$x_i \rightarrow (k_1, k_2, \dots, k_L).$$

These indices are then projected into continuous vector representations using an embedding matrix:

$$E \in \mathbb{R}^{V \times d_e},$$

where d_e is the embedding dimension. The embedding lookup produces a sequence of dense vectors:

$$(\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_L), \quad \mathbf{e}_t \in \mathbb{R}^{d_e},$$

which captures semantic similarity between words and serves as the input to sequence models such as BiLSTM and transformer-based architectures.

5.3 Classification Models

This subsection describes the mathematical formulation and learning principles of each classifier evaluated in this study.

5.3.1 Logistic Regression

Logistic Regression is used as a linear probabilistic baseline model. Given an input feature vector $\mathbf{x}$, the model computes a linear score for each class and converts it into a probability distribution using the softmax function:

$$P(y = c \mid \mathbf{x}) = \frac{e^{\mathbf{w}_c^\top \mathbf{x} + b_c}}{\sum_{k=1}^C e^{\mathbf{w}_k^\top \mathbf{x} + b_k}},$$

where $\mathbf{w}_c$ and b_c are the weight vector and bias for class c , and $C = 3$ is the number of classes. The parameters are learned by minimizing the categorical cross-entropy loss:

$$\mathcal{L} = - \sum_{i=1}^N \sum_{c=1}^C \mathbb{I}(y_i = c) \log P(y = c \mid \mathbf{x}_i),$$

which corresponds to maximum likelihood estimation under a multinomial logistic model. This classifier provides a strong and interpretable linear baseline for text classification.

5.3.2 Multinomial Naïve Bayes

The Multinomial Naïve Bayes classifier is a generative model that estimates class-conditional word distributions under the conditional independence assumption:

$$P(y \mid \mathbf{x}) \propto P(y) \prod_{j=1}^d P(x_j \mid y),$$

where x_j denotes the j -th feature component of the TF-IDF vector. The predicted class is obtained by:

$$\hat{y} = \arg \max_y P(y) \prod_{j=1}^d P(x_j \mid y).$$

Although the independence assumption is simplistic, this model is computationally efficient, robust to high-dimensional sparse features, and often performs surprisingly well in text classification tasks.

5.3.3 Support Vector Machine

A linear Support Vector Machine (SVM) aims to find a hyperplane that maximizes the margin between classes. The decision function is:

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b.$$

Training involves solving the following convex optimization problem:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^N \xi_i,$$

subject to:

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0,$$

where ξ_i are slack variables allowing soft-margin violations and C controls the trade-off between margin maximization and classification error. Multi-class classification is implemented using a one-vs-rest strategy. The large-margin principle generally leads to strong generalization performance in high-dimensional spaces.

5.3.4 Random Forest

Random Forest is an ensemble learning method that combines multiple decision trees:

$$\hat{y} = \text{mode}\{T_1(\mathbf{x}), \dots, T_M(\mathbf{x})\},$$

where $T_m(\cdot)$ denotes the m -th decision tree. Each tree is trained on a bootstrap sample of the training data, and at each split, a random subset of features is considered. This randomness reduces correlation among trees, thereby lowering variance and improving generalization performance.

5.3.5 XGBoost

XGBoost is a gradient boosting framework that builds an additive model of decision trees:

$$\hat{y}_i = \sum_{k=1}^K f_k(\mathbf{x}_i), \quad f_k \in \mathcal{F},$$

where each f_k represents a regression tree. The objective function is defined as:

$$\mathcal{L} = \sum_{i=1}^N \ell(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k),$$

with regularization term:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2,$$

where T is the number of leaves in the tree and w denotes the leaf weights. The regularization term penalizes model complexity, helping to prevent overfitting while maintaining high predictive accuracy.

5.3.6 Bidirectional LSTM

The Bidirectional Long Short-Term Memory (BiLSTM) network is designed to model sequential dependencies in text from both past and future contexts. Given embedded inputs $(\mathbf{e}_1, \dots, \mathbf{e}_L)$, the forward and backward LSTMs compute:

$$\vec{\mathbf{h}}_t = \text{LSTM}_f(\mathbf{e}_t, \vec{\mathbf{h}}_{t-1}), \quad \overleftarrow{\mathbf{h}}_t = \text{LSTM}_b(\mathbf{e}_t, \overleftarrow{\mathbf{h}}_{t+1}).$$

The hidden states are concatenated:

$$\mathbf{h}_t = [\vec{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t],$$

and aggregated (e.g., by pooling) into a fixed-length sentence representation $\mathbf{h}$. This vector is passed to a softmax classifier:

$$P(y | x) = \text{softmax}(W\mathbf{h} + b).$$

By processing the sequence in both directions, the BiLSTM captures long-range contextual dependencies that are not accessible to unidirectional models.

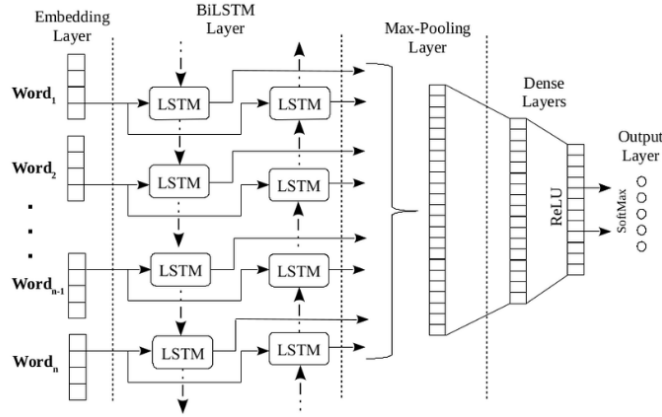


Figure 7: Bidirectional LSTM Architecture

5.3.7 BERT

BERT is a transformer-based model built upon multi-head self-attention mechanisms. The scaled dot-product attention is defined as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V,$$

where Q , K , and V denote the query, key, and value matrices, and d_k is the key dimension used for scaling. Through stacked attention layers, BERT learns deep contextual representations of input sequences. For classification, the contextual embedding corresponding to the special [CLS] token, denoted by $\mathbf{h}_{CLS}$, is used:

$$P(y | x) = \text{softmax}(W\mathbf{h}_{CLS} + b).$$

The entire network is fine-tuned using cross-entropy loss, enabling task-specific adaptation while leveraging large-scale pretraining.

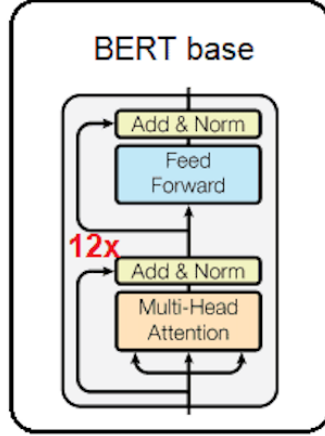


Figure 8: BERT Architecture

5.4 Training Procedure

All models are trained using the same training set and evaluated on the same held-out test set to ensure fair comparison. Classical models are optimized using convex optimization or greedy tree-growing procedures. Neural models are trained using stochastic gradient descent with backpropagation. Regularization techniques such as early stopping and weight decay are applied to mitigate overfitting and improve generalization.

5.5 Evaluation Metrics

For each class c , precision, recall, and F1-score are defined as:

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}, \quad \text{Recall}_c = \frac{TP_c}{TP_c + FN_c},$$

$$\text{F1}_c = \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}.$$

Macro-averaged F1-score is computed as:

$$\text{F1}_{macro} = \frac{1}{C} \sum_{c=1}^C \text{F1}_c,$$

and overall accuracy is defined as:

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(\hat{y}_i = y_i).$$

Macro-averaging assigns equal importance to each class, making it particularly suitable for evaluating multi-class problems with potentially imbalanced class distributions.

6 Results

This section presents the experimental results obtained from evaluating classical machine learning, ensemble, and deep learning models on the three-class spam comment dataset consisting of *ham*, *spam*, and *smishing* categories. All models were trained using an identical preprocessing pipeline and evaluated on a held-out test set of 2039 samples. Performance is reported using Accuracy, Macro-averaged Precision, Recall, F1-score, and Macro ROC-AUC (where applicable), along with class-wise Precision, Recall, and F1-scores.

6.1 Overall Performance Comparison

Table 1 summarizes the overall performance of all evaluated models.

Table 1: Overall performance comparison of different models on the three-class classification task

Model	Accuracy	Precision (Macro)	Recall (Macro)	F1-score (Macro)	ROC-AUC (Macro)
Logistic Regression	0.9691	0.9691	0.9691	0.9691	0.9981
Multinomial Naïve Bayes	0.9598	0.9600	0.9598	0.9599	0.9887
Linear SVM	0.9804	0.9805	0.9804	0.9804	–
Random Forest	0.9799	0.9799	0.9799	0.9799	0.9989
XGBoost	0.9716	0.9716	0.9716	0.9716	0.9984
BiLSTM	0.9828	0.9829	0.9828	0.9828	–
BERT	0.9774	0.9777	0.9774	0.9775	–

All models achieve strong performance, with accuracies ranging from 95.98% to 98.09%. The highest accuracy and macro F1-score are obtained by the BiLSTM model (98.09% accuracy, 0.9809 macro F1), closely followed by Linear SVM (98.04%) and Random Forest (97.99%). The transformer-based BERT model achieves an accuracy of 97.74%, demonstrating competitive performance. Classical approaches such as Logistic Regression and Multinomial Naïve Bayes also perform strongly, confirming the effectiveness of TF-IDF-based representations for this task.

6.2 Class-wise Performance Analysis

To provide a detailed view of model behavior across categories, Tables 2–8 report the class-wise Precision, Recall, and F1-scores for each model.

Logistic Regression As shown in Table 2, Logistic Regression achieves balanced performance across all three classes, with F1-scores of 0.9883 for ham, 0.9598 for smishing, and 0.9591 for spam. The model demonstrates strong discrimination between benign and malicious content, reflected in its high recall for ham (0.9941) and competitive precision for both malicious categories. The slightly lower F1-scores for smishing and spam indicate residual confusion between these two classes, which is expected due to overlapping lexical patterns. Overall, the results confirm that a linear classifier combined with TF-IDF features provides a strong and interpretable baseline for short message classification.

Table 2: Class-wise performance of Logistic Regression

Class	Precision	Recall	F1-score	Support
Ham	0.9825	0.9941	0.9883	679
Smishing	0.9714	0.9485	0.9598	680
Spam	0.9535	0.9647	0.9591	680

Multinomial Naïve Bayes The class-wise results in Table 3 show that Multinomial Naïve Bayes performs particularly well on the ham class, achieving an F1-score of 0.9852. However, its performance on smishing (0.9497) and spam (0.9447) is comparatively lower, indicating increased confusion between the two malicious categories. This behavior can be attributed to the model’s conditional independence assumption, which limits its ability to capture contextual dependencies and subtle semantic differences in short messages. Despite this limitation, the model remains computationally efficient and provides competitive performance in high-dimensional sparse feature spaces.

Table 3: Class-wise performance of Multinomial Naïve Bayes

Class	Precision	Recall	F1-score	Support
Ham	0.9911	0.9794	0.9852	679
Smishing	0.9421	0.9574	0.9497	680
Spam	0.9468	0.9426	0.9447	680

Linear SVM Table 4 reports that the Linear SVM achieves near-perfect performance on the ham class, with an F1-score of 0.9978, and strong results on smishing (0.9718) and spam (0.9716). The high precision and recall values across all classes indicate effective margin-based separation in the high-dimensional TF-IDF feature space. Notably, the model exhibits very low confusion between benign and malicious messages, which is critical for security-sensitive applications. These results highlight the strength of large-margin classifiers for robust text classification tasks.

Table 4: Class-wise performance of Linear SVM

Class	Precision	Recall	F1-score	Support
Ham	0.9971	0.9985	0.9978	679
Smishing	0.9805	0.9632	0.9718	680
Spam	0.9638	0.9794	0.9716	680

Random Forest As shown in Table 5, Random Forest demonstrates consistently strong performance across all categories, with F1-scores of 0.9978 for ham, 0.9714 for smishing, and 0.9705 for spam. The ensemble approach enables the model to capture nonlinear feature interactions and reduces variance through averaging over multiple decision trees. The balanced precision and recall values suggest robust generalization, while the remaining errors are primarily due to semantic overlap between smishing and spam messages rather than systematic model bias.

Table 5: Class-wise performance of Random Forest

Class	Precision	Recall	F1-score	Support
Ham	0.9971	0.9985	0.9978	679
Smishing	0.9693	0.9735	0.9714	680
Spam	0.9734	0.9676	0.9705	680

XGBoost The class-wise performance in Table 6 indicates that XGBoost maintains balanced results across all three classes, achieving F1-scores of 0.9897 for ham, 0.9661 for smishing, and 0.9589 for spam. While the model performs strongly overall, its slightly lower scores on the malicious categories suggest sensitivity to overlapping lexical and semantic patterns. The gradient boosting framework effectively refines decision boundaries through iterative error correction, but the results show that the gains over other ensemble methods are marginal for this dataset.

Table 6: Class-wise performance of XGBoost

Class	Precision	Recall	F1-score	Support
Ham	0.9897	0.9897	0.9897	679
Smishing	0.9676	0.9647	0.9661	680
Spam	0.9575	0.9603	0.9589	680

BiLSTM According to Table 7, the BiLSTM model achieves the best overall performance among all evaluated methods, with an accuracy of 98.28% and a macro-averaged F1-score of 0.9828. The model attains near-perfect performance on the ham class ($F1 = 0.9993$) and strong, well-balanced results on smishing ($F1 = 0.9742$) and spam ($F1 = 0.9750$). These results demonstrate the effectiveness of bidirectional sequence modeling in capturing contextual dependencies within short messages and in distinguishing subtle differences between malicious categories. Figure 9 shows the accuracy curve and Figure 10 shows the Loss curve of the BiLSTM Model.

Table 7: Class-wise performance of BiLSTM

Class	Precision	Recall	F1-score	Support
Ham	1.0000	0.9985	0.9993	679
Smishing	0.9750	0.9735	0.9742	680
Spam	0.9736	0.9765	0.9750	680

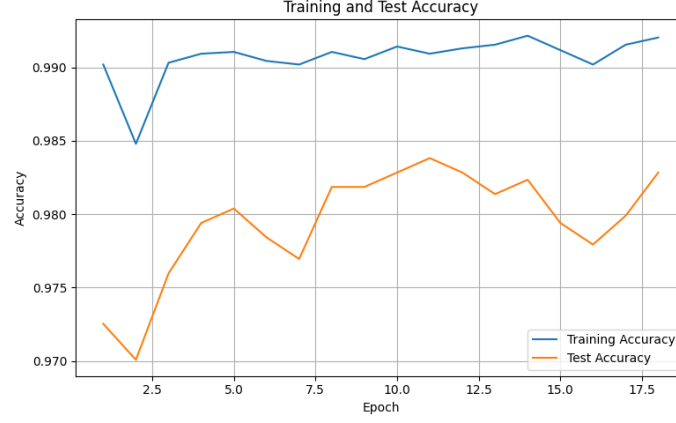


Figure 9: Accuracy curve of BiLSTM



Figure 10: Loss Curve of BiLSTM

BERT The results in Table 8 show that BERT achieves strong and consistent performance across all classes, with an F1-score of 0.9956 for ham, 0.9697 for smishing, and 0.9673 for spam. The perfect precision observed for the ham class indicates highly reliable detection of benign messages. Although BERT leverages rich contextual representations through self-attention, the performance gains over lighter models are relatively modest, suggesting that the dataset size and short message length may limit the full advantage of large pretrained transformer architectures in this setting.

Table 8: Class-wise performance of BERT

Class	Precision	Recall	F1-score	Support
Ham	1.0000	0.9912	0.9956	679
Smishing	0.9762	0.9632	0.9697	680
Spam	0.9568	0.9779	0.9673	680

6.3 Confusion Matrix Analysis

Figures 11–17 illustrates the confusion matrices obtained for all evaluated models. Across all experiments, the matrices exhibit strong diagonal dominance, indicating robust classification performance for the three classes: ham, smishing, and spam.

Logistic Regression The Logistic Regression classifier exhibits well-balanced performance across all three classes, with strong diagonal dominance in the confusion matrix. Out of 679 ham messages, 675 are correctly classified, indicating excellent discrimination between benign and malicious content. Misclassifications are primarily observed between smishing and spam, with 29 smishing messages incorrectly labeled as spam and 18 spam messages classified as smishing. Crucially, only six malicious messages in each class are misclassified as ham, demonstrating that the model maintains a low false-negative rate for security-critical detections.

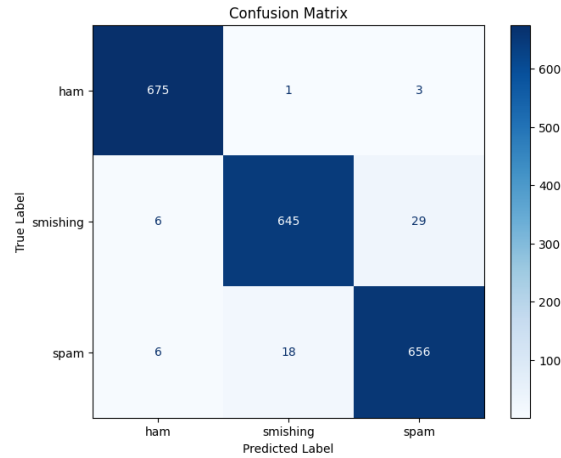


Figure 11: Confusion matrix of Logistic Regression

Multinomial Naïve Bayes Multinomial Naïve Bayes shows comparatively higher cross-class confusion, particularly between spam and smishing. While ham detection remains strong (665 out of 679 correctly classified), 34 spam messages are misclassified as smishing, and 28 smishing messages are labeled as spam. This increased overlap can be attributed to the model's conditional independence assumption, which limits its ability to model contextual dependencies and subtle semantic cues within malicious messages.

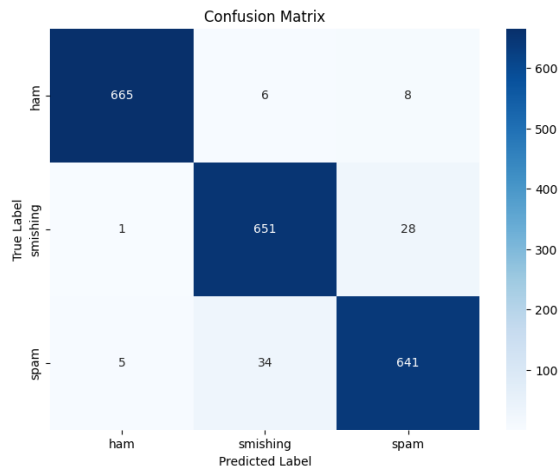


Figure 12: Confusion matrix of Multinomial Naive Bayes

Linear SVM The Linear SVM produces one of the cleanest confusion matrices among all evaluated models. Ham classification is near perfect, with 678 correct predictions and only one misclassification. Notably, the model makes no smishing-to-ham errors, underscoring its strong margin-based separation. Residual errors are largely confined to smishing and spam, with 25 smishing messages mislabeled as spam. These results highlight the effectiveness of large-margin classifiers in high-dimensional TF-IDF feature spaces.

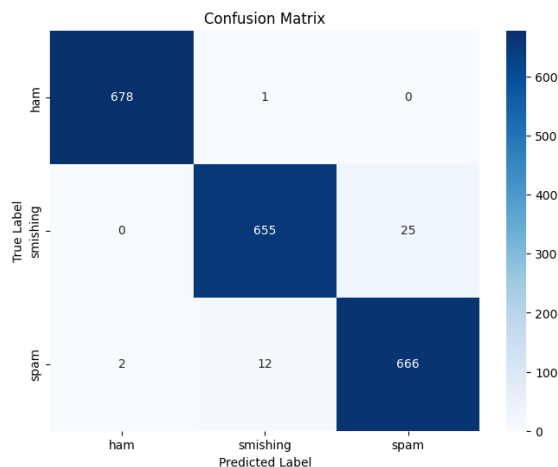


Figure 13: Confusion matrix of Linear SVM

Random Forest Random Forest demonstrates robust and consistent performance across all classes. Similar to Linear SVM, ham classification is nearly flawless, with only one misclassification. The ensemble correctly identifies 662 smishing and 658 spam messages, with remaining errors largely due to confusion between smishing and spam. The model's ability to capture nonlinear feature interactions through ensemble learning contributes to its strong generalization performance.

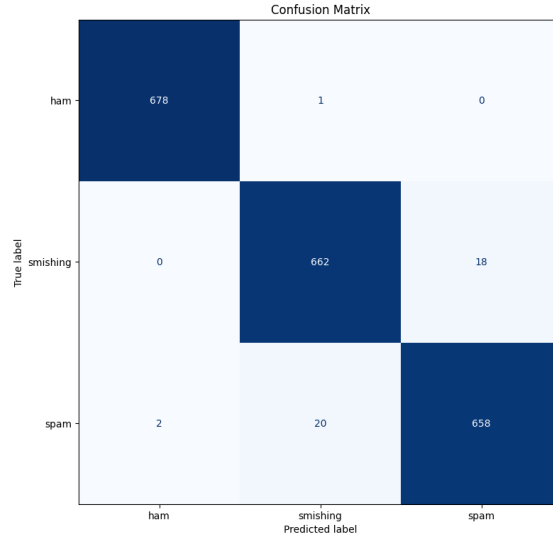


Figure 14: Confusion matrix of Random Forest

XGBoost XGBoost achieves high overall accuracy but exhibits slightly elevated confusion between malicious classes compared to Random Forest and Linear SVM. Specifically, 24 smishing messages are misclassified as spam, and 20 spam messages are labeled as smishing. Additionally, a small number of ham messages (seven) are incorrectly classified as spam. These patterns suggest that while gradient boosting refines decision boundaries effectively, it remains sensitive to overlapping lexical and semantic features.

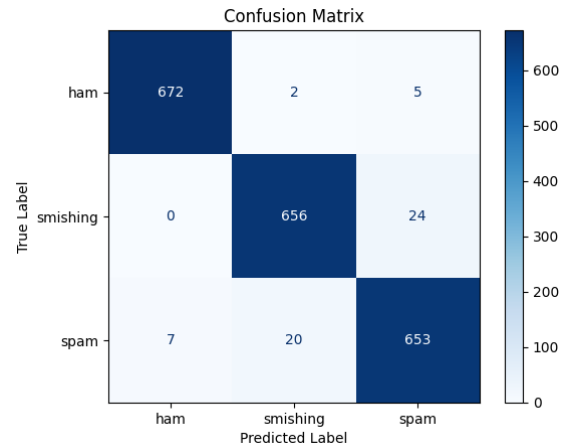


Figure 15: Confusion matrix of XGBoost

BiLSTM The BiLSTM model demonstrates excellent sequence-level learning, yielding a confusion matrix comparable to that of the best classical models. Ham messages are classified with near-perfect accuracy (678 correct predictions), and no smishing messages are misclassified as ham. The majority of errors occur between smishing and spam, with 24 smishing messages mislabeled as spam and 12 spam messages classified as smishing. This behavior reflects inherent semantic similarity rather than architectural limitations.

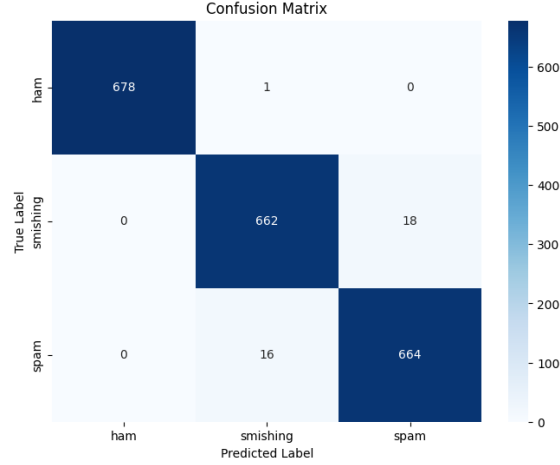


Figure 16: Confusion matrix of BiLSTM

BERT BERT achieves strong overall performance with perfect precision for ham messages, as no ham samples are incorrectly classified as smishing. The confusion matrix indicates that 25 smishing messages are mislabeled as spam, and 15 spam messages are classified as smishing. While the transformer-based architecture captures rich contextual information, the marginal gains over lighter models suggest that dataset size and message length may limit the full advantage of large pretrained representations.

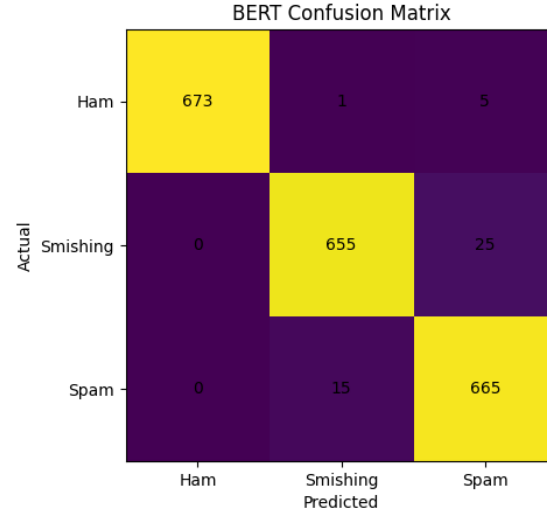


Figure 17: Confusion matrix of BERT

Overall, confusion matrix analysis reveals that smishing and spam remain the most difficult classes to distinguish, while ham detection is consistently robust across all models.

6.4 Summary of Findings

Overall, deep learning models and large-margin classifiers achieve the strongest performance, with BiLSTM, Linear SVM, and Random Forest providing the highest accuracy and macro F1-scores. These models demonstrate strong generalization and robust class separation, particularly for the benign *ham* category and malicious message detection.

The analysis of ROC curves further corroborates these findings. All evaluated models exhibit high macro ROC-AUC values, indicating excellent discriminative capability across the three classes. Linear SVM, Random Forest, and BiLSTM

achieve near-perfect ROC behavior, reflecting well-separated decision boundaries. Notably, Logistic Regression and XGBoost also attain macro ROC–AUC values above 0.99, highlighting that classical models with appropriate feature engineering can perform competitively even against deep learning approaches.

Multinomial Naïve Bayes, although slightly lower in overall accuracy, maintains a strong ROC profile, demonstrating reliable probabilistic discrimination despite its simplifying assumptions. Overall, the ROC curve analysis confirms that both classical and deep learning models are effective for spam and smishing detection, with performance differences arising primarily from fine-grained class overlap rather than fundamental limitations in model capacity.

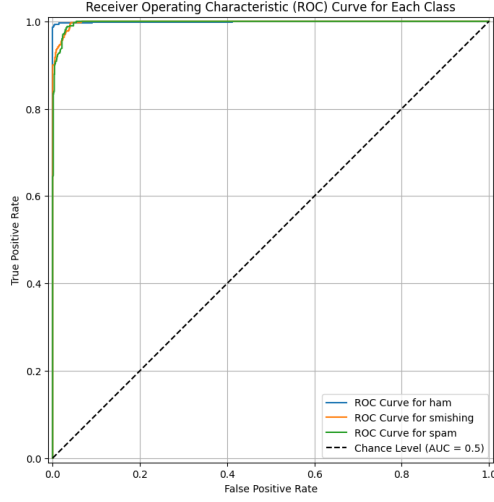


Figure 18: ROC curves of Logistic Regression

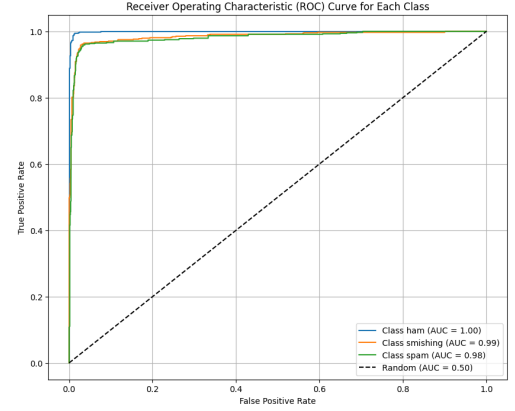


Figure 19: ROC curves of Multinomial Naïve Bayes

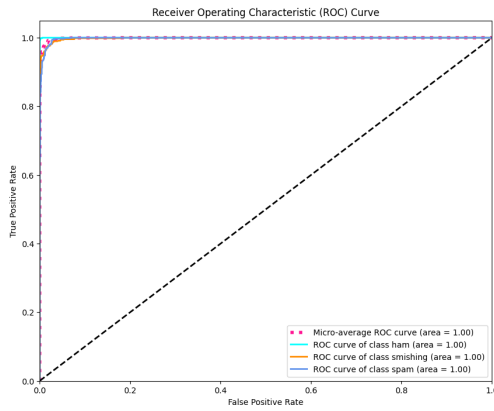


Figure 20: ROC curves of Random Forest

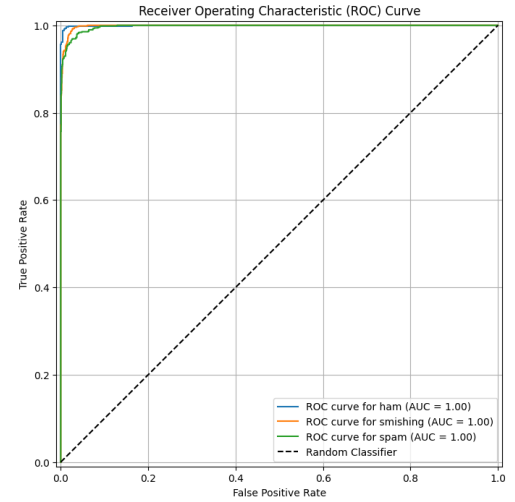


Figure 21: ROC curves of XGBoost

7 Discussion

The experimental results show that all evaluated models perform strongly on the three-class classification task (*ham*, *spam*, and *smishing*), indicating that the preprocessing pipeline and feature representations are effective. Classical models such as Logistic Regression and Multinomial Naïve Bayes provide competitive baselines, with Logistic Regression achieving well-balanced performance across classes, while Naïve Bayes shows slightly reduced performance

on *spam* and *smishing*, likely due to its conditional independence assumption and overlapping word distributions in malicious messages. Among traditional and ensemble methods, Linear SVM and Random Forest achieve near-optimal results, highlighting the strength of large-margin classifiers and ensemble learning in high-dimensional text feature spaces. XGBoost also maintains stable and balanced performance across all categories.

Deep learning models further confirm the importance of contextual and sequential information in message classification. The BiLSTM achieves the highest macro-averaged F1-score, demonstrating the benefit of modeling word order and long-range dependencies, which are particularly useful for distinguishing between different types of malicious messages. The BERT model also performs consistently well across all classes, especially on the *ham* class, showing the effectiveness of pretrained transformer representations. However, the performance gap between deep models and strong classical models remains small, suggesting that the task is already close to saturation with well-engineered features and that dataset size and complexity may limit further gains from heavier architectures.

Analysis of the confusion matrices reveals that most misclassifications occur between the *spam* and *smishing* classes, while *ham* is detected with very high precision and recall across all models. This indicates that the primary challenge lies in fine-grained discrimination between different types of malicious content rather than separating benign from malicious messages. Overall, the results suggest a practical trade-off between model complexity and performance: while deep learning models offer strong representational power, simpler models such as Linear SVM and Random Forest remain highly competitive, making them attractive choices for real-world, resource-constrained deployments.

8 Conclusion

This study presented a comprehensive comparison of classical machine learning, ensemble, and deep learning models for three-class message classification into *ham*, *spam*, and *smishing*. Using a unified preprocessing pipeline and a consistent evaluation protocol, all models achieved strong performance, demonstrating that the chosen feature representations and learning strategies are well-suited for this task. Among the evaluated approaches, Linear SVM, Random Forest, and BiLSTM achieved the highest overall performance, while BERT also showed robust and consistent results across all classes. Classical models such as Logistic Regression and Multinomial Naïve Bayes remained competitive, highlighting that simpler methods can still be highly effective when combined with appropriate text preprocessing.

The class-wise and confusion matrix analyses revealed that most errors occur between the *spam* and *smishing* categories, whereas the *ham* class is detected with very high reliability across all models. This indicates that the main challenge is not in separating benign and malicious messages, but in distinguishing between different types of malicious content. The results further suggest that while deep learning models benefit from contextual and sequential representations, the performance gains over strong traditional models are relatively small for this dataset, pointing to a possible saturation effect given the current data scale and complexity.

Future work can focus on expanding the dataset, incorporating more fine-grained semantic features, and exploring advanced transformer-based architectures or domain-adaptive pretraining to further improve discrimination between closely related malicious classes. Additionally, evaluating these models in real-world deployment settings, considering factors such as inference time, memory usage, and robustness to adversarial or evolving message patterns, would provide valuable insights for building practical and scalable spam and smishing detection systems.

References

- [1] M. Munoz and M. Islam, “A balanced dataset for spam and smishing detection using large language models (llms),” 2025.
- [2] A. Gupta and R. Kaushal, “Improving spam detection in online social networks,” *Procedia Computer Science*, vol. 132, pp. 373–379, 2018.
- [3] B. Sanz, I. Santos, C. Laorden, and P. G. Bringas, “On the automatic categorization of android applications,” *Computers & Security*, vol. 40, pp. 1–17, 2014.
- [4] X. Zhang, J. Zhao, and Y. LeCun, “Character-level convolutional networks for text classification,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015.
- [5] S. Minaee, N. Kalchbrenner, E. Cambria, N. Nikzad, M. A. Chenaghlu, and J. Gao, “Deep learning–based text classification: A comprehensive review,” *ACM Computing Surveys*, vol. 54, no. 3, pp. 1–40, 2021.
- [6] S. K. Bhowmick, S. Barman, S. Das, A. Mandal, and M. Shahil, “Redefining cyberbullying detection with kolmogorov-arnold networks (kans) for building interpretable ai beyond black-box mlps,” in *2025 IEEE Guwahati Subsection Conference (GCON)*, 2025, pp. 1–6.

- [7] S. K. Bhowmick, S. Barman, and S. K. Biswas, “A multidimensional approach to context-aware cyberbullying detection through metadata analysis,” in *Intelligent Computing and Technologies*, ser. Communications in Computer and Information Science, P. P. Singh, S. Prasad, P. K. Singh, and P. Radeva, Eds. Cham: Springer, 2026, vol. 2717.
- [8] M. T. Ribeiro, S. Singh, and C. Guestrin, ““why should i trust you?”: Explaining the predictions of any classifier,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016, pp. 1135–1144.
- [9] F. Doshi-Velez and B. Kim, “Towards a rigorous science of interpretable machine learning,” *arXiv preprint arXiv:1702.08608*, 2017.
- [10] T. A. Almeida, J. M. Gómez Hidalgo, and A. Yamakami, “Contributions to the study of sms spam filtering: New collection and results,” *Proceedings of the 11th ACM Symposium on Document Engineering*, pp. 259–262, 2011.
- [11] T. C. Alberto, J. V. Lochter, and T. A. Almeida, “Tubespam: Comment spam filtering on youtube,” in *Proceedings of the IEEE International Conference on Machine Learning and Applications (ICMLA)*, 2015.
- [12] R. K. Das, S. S. Dash, K. Das, and M. Panda, “Detection of spam in youtube comments using different classifiers,” in *Advanced Computing and Intelligent Engineering*. Springer, 2020.
- [13] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [14] M. Schuster and K. K. Paliwal, “Bidirectional recurrent neural networks,” *IEEE Transactions on Signal Processing*, vol. 45, no. 11, pp. 2673–2681, 1997.
- [15] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [16] T. Chen and C. Guestrin, “Xgboost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [17] N. Hussain *et al.*, “Spam review detection techniques: A systematic literature review,” *Applied Sciences*, vol. 9, no. 5, p. 987, 2019.
- [18] A. R. Chrismanto, A. K. Sari, and Y. Suyanto, “Critical evaluation on spam content detection in social media,” *Journal of Theoretical and Applied Information Technology*, vol. 100, no. 8, pp. 2642–2667, 2022.
- [19] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of NAACL-HLT*, 2019.